

Ch01 軟體危機、品質模型與 AI 時代的可靠性工程

Chapter 01: The Software Crisis, Quality Models, and AI-Era Reliability Engineering

😄 大家都知道「物質不滅定律」；身為資工系學生，我們更熟悉「Bug 不滅定律」。

在 2026 年，寫出一段程式碼只要問 AI 3 秒鐘；但要證明這段程式碼在生產環境不會搞垮公司，可能要花上 3 個月。

1.1 軟體危機的歷史與 AI 時代的輪迴

軟體既能造福人類，亦能造成毀滅性災難。回顧歷史，軟體缺陷曾引發嚴重的空難、軍事傷亡與數億美元的太空浩劫。

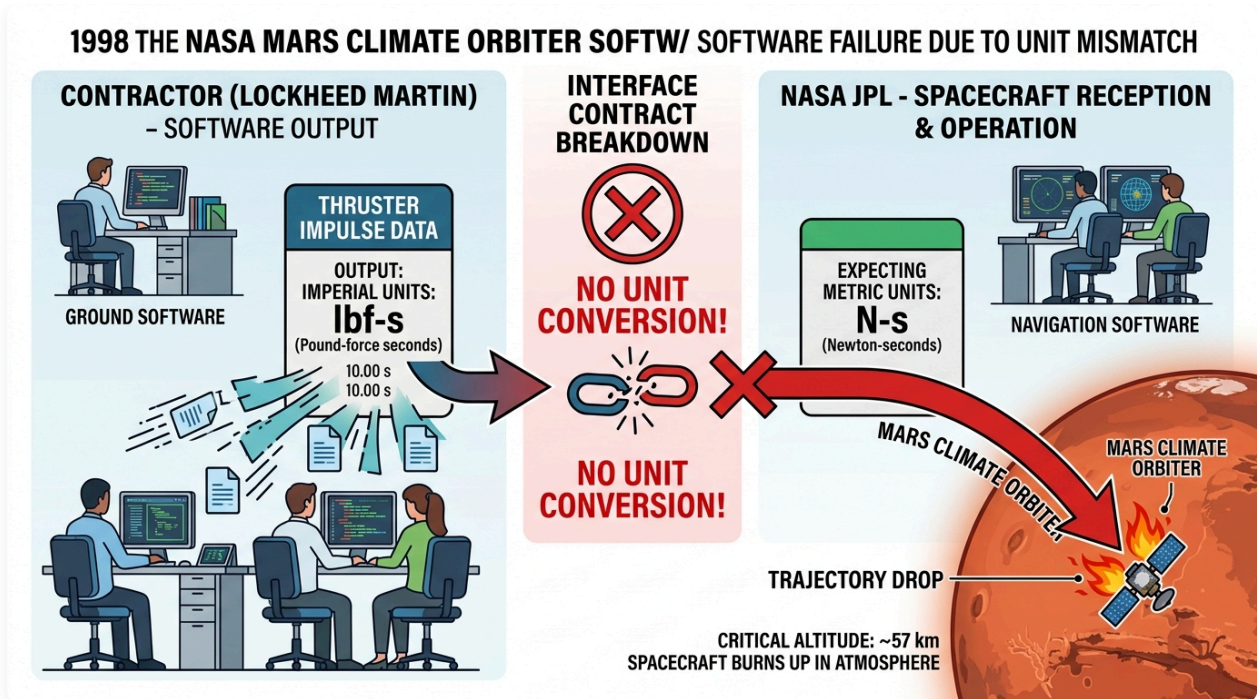
1.1.1 Case 1：愛國者反導彈事件 (1991) —— 毫秒級的精度累積誤差

在 1991 年 2 月波斯灣戰爭中，一枚伊拉克發射的飛毛腿飛彈擊中美軍沙烏地達蘭基地，造成 28 名美軍死亡、100 多人受傷。

- 致命軟體缺陷**：愛國者系統時鐘暫存器採用 **24-bit 浮點數** 設計，將時間轉換為 0.1 秒單位時產生了微小的截斷誤差（約 0.000000095 秒）。
- 災難放大**：雷達系統連續開機運作超過 **100 小時** 未重啟，微小的精度誤差累計達 **0.33 秒**。
- 致命後果**：飛毛腿飛彈速度達 4.2 馬赫（1.5 km/s），0.33 秒相當於 **600 公尺距離偏差**。雷達搜尋窗無法鎖定目標，攔截飛彈根本沒有發射。
- SQA 啟示**：數值精度問題、浮點數累計誤差，以及**長時運行可靠度測試（Long-term Stress/Reliability Testing）**的重要性。

1.1.2 Case 2：NASA 火星氣候軌道探測器 (1998) —— 單位的代價

1998 年 NASA 發射「火星氣候軌道探測器」（Mars Climate Orbiter，造價近 2 億美元），抵達火星後失聯焚毀。



圖形解說：跨模組介面契約斷裂導致太空船墜毀

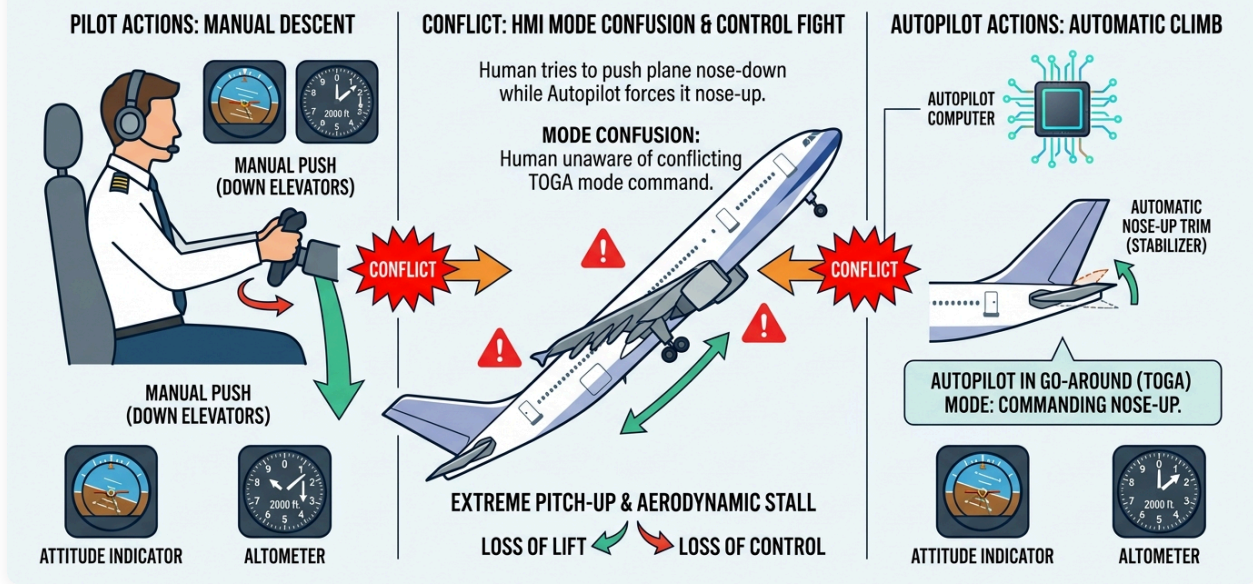
- 【左側】承包商軟體端（洛克希德馬丁）：地面控制程式以 英制單位（磅力·秒，lbf·s）輸出推進器衝量數據。
- 【中間】介面契約斷裂 (Interface Contract Breakdown)：兩端系統缺乏嚴謹的介面型態定義與自動化單位轉換機制。
- 【右側】NASA JPL 導航接收端：太空船導航軟體預設以 公制單位（牛頓·秒，N·s）解析輸入數據，導致推力計算出現 4.45 倍的嚴重偏差。
- 災難後果：軌道高度預計 140 公里，實際暴跌至 57 公里，直接在火星大氣層中摩擦燃燒解體。
- SQA 啟示：跨模組介面契約 (Interface Contract)、強型態檢驗與規格審查的重要性。

1.1.3 Case 3：華航名古屋空難 (1994) —— 人機爭奪控制權

1994 年 4 月 26 日，華航 CI140 班機（空中巴士 A300-622R）在名古屋機場降落時墜毀，264 人罹難。

1994 CHINA AIRLINES NAGOYA CRASH: FLIGHT CONTROL SOFTWARE CONFLICT (HMI MODE CONFUSION)

Case Study: Human-Machine Interface Design & System Safety



圖形解說：人機介面衝突 (HMI Mode Confusion) 與控制權仲裁缺失

- **【左側】機師手動操作 (Manual Push)：**副駕駛誤觸「重飛 (Go-Around / TOGA)」模式後，正副駕駛試圖手動前推操縱桿 (Down Elevators) 強壓機首下降以利降落。
- **【右側】飛控電腦自動配平 (Autopilot Automatic Climb)：**機載飛控電腦因處於「自動重飛」模式，強行將水平安定面 (Horizontal Stabilizer) 向上配平以抬高機首爬升。
- **【中央衝突】模式混淆與控制權爭奪 (Control Fight)：**駕駛員未察覺電腦仍在執行重飛指令，人機力量相互抵消；最終水平安定面達到極限仰角，飛機在低空發生氣動失速 (Aerodynamic Stall) 墜毀。
- **SQA 啟示：**人機互動 (HMI/UX) 狀態透明度、異常操作回饋與自動化控制權限仲裁設計。

1.1.4 Case 4：迪士尼《獅子王》遊戲 (1994) —— 缺乏相容性測試的公關浩劫

1994 年聖誕節迪士尼推出《獅子王》PC 遊戲，伴隨 Compaq 等家用電腦熱銷，數以萬計家庭期待同樂。

- **致命缺陷：**遊戲基於特定視訊驅動 (WinG) 開發，未在市場主流多樣硬體環境上進行充分相容性測試。
- **災難後果：**大量家用電腦開機即藍屏崩潰，聖誕節當天客服被憤怒家長打爆，嚴重損害品牌聲譽。
- **SQA 啟示：**環境多樣性驗證與**相容性測試 (Compatibility Testing)**，促使微軟後來開發標準化 DirectX 架構。

1.1.5 軟體危機的定義與成因

1968 年 NATO 會議首次提出「軟體危機 (Software Crisis)」：硬體飛速發展，而軟體開發卻面臨預算超支、進度延期、品質低下、維護困難等問題。軟體危機的原因可以歸納為以下幾點：

1. 軟體複雜性：隨著電腦硬體性能的提升，軟體的規模和複雜性不斷增加，超出了傳統軟體開發方法的處理能力。
2. 軟體開發效率低下：軟體開發的進度和成本往往難以預測，開發團隊規模不斷擴大，開發效率卻沒有同步提升。
3. 軟體品質低下：軟體錯誤率高，軟體品質難以保證，導致軟體系統不穩定，甚至引發嚴重的安全事故。
4. 軟體維護困難：隨著軟體規模的擴大，軟體維護的難度不斷增加，軟體開發團隊難以適應軟體維護的需求。

概念核對問答 (CCQ 1)

問題

愛國者反導彈系統（1991）在達蘭基地攔截失效的根本軟體原因為何？

- A) 通訊網路中斷導致雷達無法傳送指令給飛彈發射架
- B) 24-bit 時鐘暫存器的浮點捨入誤差在連續運行 100 小時後累加達 0.33 秒
- C) 程式碼發生記憶體洩漏 (Memory Leak) 導致作業系統當機
- D) 雷達演算法誤將美軍戰機辨識為敵方飛毛腿飛彈

[線上作答](#)



► [點擊查看【概念核對問答】答案與解析](#)

1.2 AI 能拯救軟體危機嗎？

近年來，隨著人工智慧 (AI) 技術的突飛猛進，AI 輔助開發工具（如 GitHub Copilot、ChatGPT）的普及，似乎為軟體工程界注入了一劑強心針。然而，數據顯示這並非萬靈丹，反而可能加劇了隱藏的危機。近年的多項研究與學術調查提供了驚人的數據支持：

1. 程式碼維護性惡化：GitClear 縱向研究 (2020–2026)

- GitClear 分析了 1.5 億行程式碼的 Git Commit 數據，發現自 AI 輔助工具普及以來：
 - 程式碼重複率 (Code Duplication) 呈指數級上升。
 - 衡量程式碼重構的關鍵指標「移動行數 (Moved Lines)」大幅下降，表示工程師更少主動去重構、整理舊程式碼。
 - 程式碼流失率 (Code Churn) 顯著增高（剛寫好的程式碼在短時間內被刪除或重寫），這證明 AI 生成了大量看似可行、實則脆弱的程式碼，帶來了沈重的長期維護性債務 (Maintainability Debt) [5]。

2. 52% 的高錯誤率與「虛假安全感」：Purdue University 實證研究

- 普渡大學研究團隊評估了 ChatGPT 在回答 Stack Overflow 上的 517 個軟體工程問題時的表現：
 - 結果顯示，AI 的解答中有 52% 包含錯誤的程式碼或資訊。
 - 但更可怕的是，由於 AI 的語氣極度禮貌、條理清晰且「看似極度合理」，有高達 39.3% 的使用者依然偏好並採信了 AI 的錯誤回答。
 - 這會使工程師產生錯誤的「安全感」，未經嚴謹驗證就直接將漏洞帶入系統中 [6]。

3. 40% 的安全弱點隱患：紐約大學 (NYU) 等學術研究

- 研究人員對 AI 生成的程式碼進行自動化安全掃描（基於 Common Weakness Enumeration, CWE 標準）：結果發現，AI 在沒有特定安全提示的引導下，生成程式碼中有高達約 40% 包含已知的安全弱點（如：緩衝區溢位、SQL 注入、並發競爭危害）[7]。

1.2.1 AI 寫程式引發的品質事件

1. 幻覺套件供應鏈投毒 (Slopsquatting / Package Hallucination)

- 事故機制：LLM 在撰寫程式碼時，常會「一本正經地捏造」一個聽起來非常合理的套件名稱（例如 `crypto-validator` 或 `huggingface-cli`）。
- 釀成災難：黑客監控 AI 常出現的幻覺套件名稱後，搶先在 PyPI 或 npm 上註冊同名惡意套件。不知情的工程師直接執行 AI 給的 `pip install` 或 `npm install` 指令，導致後門程式與木馬直接植入企業開發環境與生產伺服器。研究顯示有惡意概念驗證套件在數月內被無辜開發者下載超過數萬次 [1]。

2. 亞馬遜 (Amazon) 電商系統大斷線與訂單蒸發

- 事故機制：2026 年 3 月上旬，亞馬遜電商系統遭遇嚴重故障，內部文件一度指出工程師使用 AI 寫程式工具輔助生成變更程式碼，但未經完整審查與自動化防護驗證即推上生產環境。
- 釀成災難：導致送貨與結帳邏輯出錯，引發北美訂單一度崩跌 99%，在數小時內蒸發超過 630 萬筆訂單與大量營收 [2]。

3. Vibe Coding 帶來的漏洞大爆發（以 Lovable/No-code 平台為例）

- 事故機制：非工程背景人員或初階開發者僅憑 Prompt 快速產出整套 Web 服務，缺乏程式碼審查 (Code Review) 與安全防護概念。
- 釀成災難：資安團隊針對 AI 自動生成的 1,600 多個上線應用進行掃描，發現高達 10% 以上存在嚴重的 SQL Injection、越權存取 (Broken Access Control) 或敏感資料外洩漏洞，用戶可

輕易繞過驗證直接進入管理後台 [3]。

4. 敏感金鑰與憑證直接寫死 (Hardcoded Secrets) 外洩

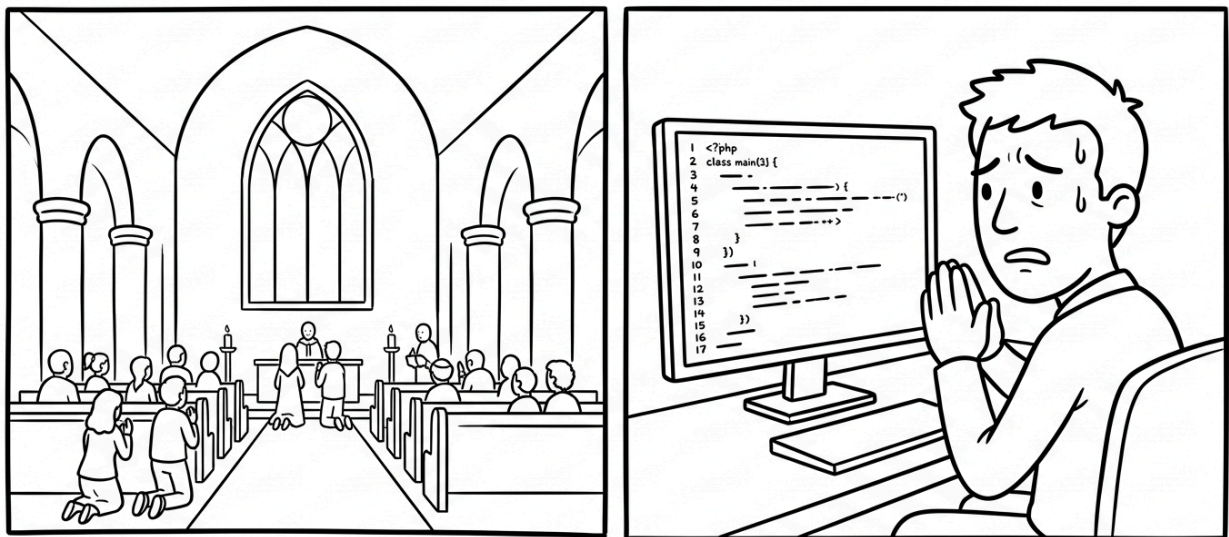
- **事故機制**：AI 為了讓程式「立刻能跑」，經常在範例程式碼中示範把 API Key、資料庫密碼寫死在程式碼內。
- **釀成災難**：開發者在沒有抽換成環境變數 (Environment Variables) 的情況下，直接將代碼推送到公開 GitHub Repository，導致雲端帳號 (如 AWS、OpenAI) 在一小時內被掃描機器人盜用並產生數萬美元的巨額帳單 [4]。

💡 結論：

在 2026 AI 時代，軟體開發的真正瓶頸 (Bottleneck) 已經從「程式碼寫不寫得出來 (Writing)」完全轉移到「程式碼到底正不正確 (Verification)」。如果開發者缺乏品質保證知識，只盲信 AI 的「綠燈完成」，將會使軟體系統迅速崩潰。

😂 軟體和教堂非常相似——建成之後我們就開始祈禱。

‘Software and cathedrals are much the same – first we build them, then we pray.’ — Sam Redwine



參考資料出處 (References) :

1. **Lasso Security: [AI Package Hallucinations](#)** — 研究指出 AI 幻覺套件 (如 `huggingface-cli`) 可能引發 Slopsquatting 攻擊，惡意套件在數月內被無辜下載超過 3 萬次。
2. **CRN: [AWS Outage Was Not AI-Caused Via Kiro Coding Tool, Amazon Confirms](#)** — 報導亞馬遜內部大推 AI 寫程式工具 Kiro 以及相關系統故障引發的代碼安全重整爭議與澄清。
3. **Threat Landscape: [Lovable.dev Data Breach: BOLA Vulnerability in Vibe Coding](#)** — 詳細分析 AI 自動建置應用平台 Lovable 於 2026 年爆發的 BOLA (IDOR) 越權漏洞與產生的程式碼/金鑰暴露風險。
4. **GitGuardian: [State of Secrets Sprawl Report 2026](#)** — 數據顯示 AI 輔助開發的金鑰與憑證洩漏率是人類開發者的兩倍 (如 Claude Code 輔助提交的洩漏率達 3.2%)。
5. **GitClear: [Coding on Copilot: 2024 Developer Research](#)** — 針對 1.5 億行程式碼進行的縱向分析，指出 AI 輔助開發使程式碼重複率與流失率增加，並降低了主動重構的頻率。
6. **Purdue University: [Is Stack Overflow Obsolete? An Empirical Study of the Characteristics of ChatGPT Answers to Stack Overflow Questions](#)** — 實證研究發現 ChatGPT 在回答軟體工程問題時，52% 的解答包含錯誤程式碼或資訊，且有 39% 的使用者採信了錯誤回答。
7. **New York University (NYU): [Asleep at the Keyboard? Assessing the Security of GitHub Copilot's Code Contributions](#)** — 學術安全掃描研究指出，在無安全提示引導下，AI 生成的程式碼中有約 40% 包含常見的安全弱點 (CWE Top 25 漏洞)。

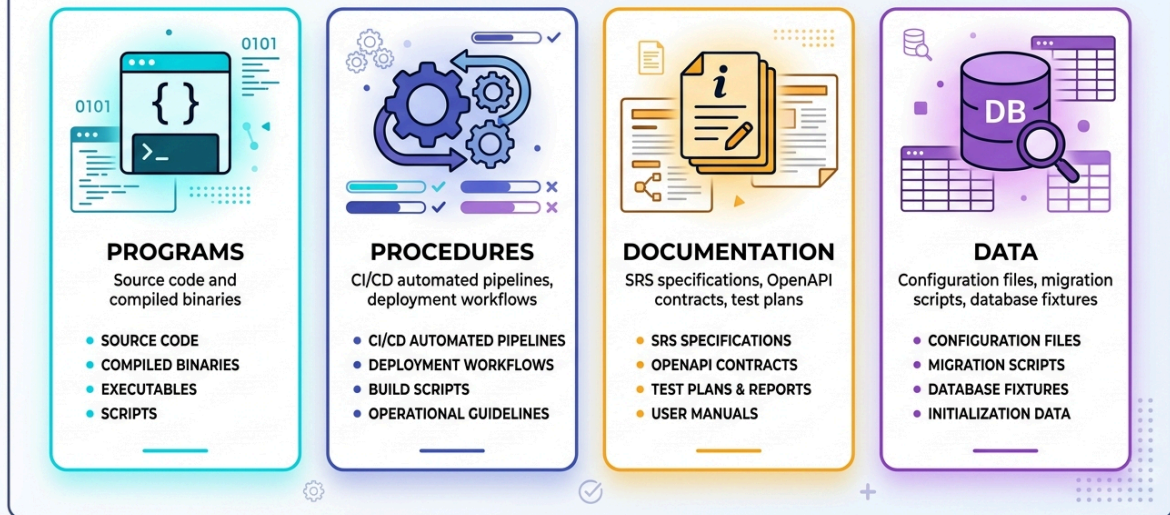
1.3 軟體的本質與品質維度 (軟體四要素 & Garvin 五大品質觀點)

1.3.1 軟體的四大組成要素 (IEEE 610.12)

軟體到底是什麼？僅僅是可執行的二進位檔案或原始程式碼嗎？根據 IEEE (Standard 610.12) 的權威定義，軟體是一個完整的系統化工程產物：

Software (軟體): Computer **programs** (程式), **procedures** (程序), and possibly associated **documentation** (文件) and **data** (資料) pertaining to the operation of a computer system.

EDUCATIONAL SOFTWARE ENGINEERING INFOGRAPHIC: 4 CORE ELEMENTS OF SOFTWARE (IEEE STANDARD)

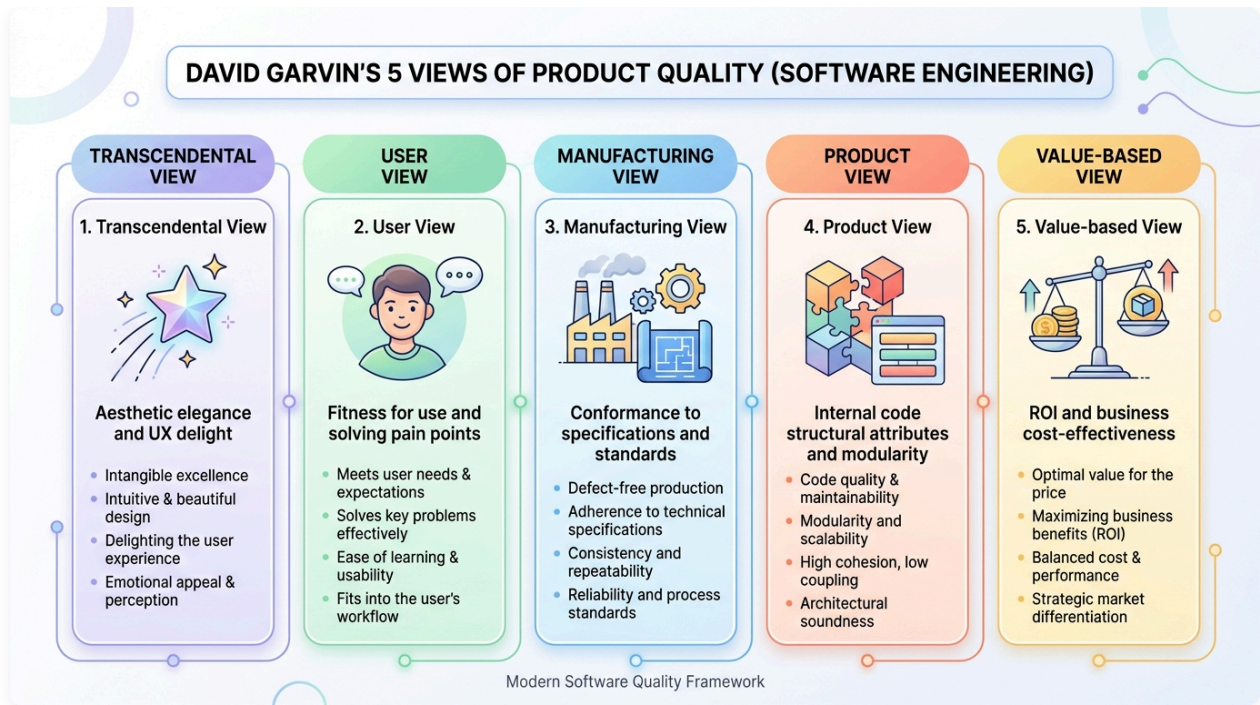


圖形解說：軟體四大核心要素 (IEEE 610.12)

1. **Programs (程式碼)**：包含原始碼 (Source Code)、編譯產物 (Bytecode/Binary) 與執行腳本，負責承載業務邏輯與演算法。
2. **Procedures (作業程序)**：包含 CI/CD 自動化建置腳本、部署規程、維運手冊 (Runbooks) 與版本發布規範。
3. **Documentation (文件與規格)**：包含需求規格書 (SRS)、OpenAPI 介面契約、架構設計圖與測試計畫（規格即活文件）。
4. **Data (資料與配置)**：包含資料庫初始化結構 (Migration)、環境變數配置設定檔與測試測資集 (Test Fixtures)。

1.3.2 何謂品質？David Garvin 的五大品質觀點

當我們探討「軟體品質」時，哈佛商學院教授 David Garvin 在《Managing Quality》中指出，品質並非單一維度，而是由多重視角交織而成的立體概念：



圖形解說：David Garvin 五大品質觀點

1. **超自然觀點 (Transcendental View)**：無法精確量化，但一體驗就能感受到其精緻、優雅與直覺的極致美感（如絲滑流暢的 UI/UX 與微互動）。
2. **使用者觀點 (User View - Fitness for Use)**：軟體是否能切中真實使用者的痛點、滿足業務需求並帶來實質效益（合用性）。
3. **製造觀點 (Manufacturing View - Conformance)**：軟體產出物與工程流程是否 100% 符合規格、通過靜態檢測與 Quality Gate（符合度）。
4. **產品觀點 (Product View - Architecture)**：產品內在結構特性，如高內聚低耦合、強固型態、可測試性與可維護性。
5. **價值觀點 (Value-based View - ROI)**：軟體帶來的商業價值與產出是否顯著高於其開發、測試與維運之總成本（投資報酬率）。

品質觀點	核心定義	軟體工程實例	忽略該觀點的後果
超自然觀點 (Transcendental)	無法精確量化，但一體 驗就能感受到其精緻與 美感	極致流暢的 UI/UX、細膩的動畫 微互動	軟體感覺粗製濫 造、冰冷難用
使用者觀點 (User View)	符合使用者真實需求與 期望 (Fitness for Use)	解決使用者痛點、 操作直覺易上手	功能很強但沒人想 用 (Shelfware)
製造觀點 (Manufacturing View)	符合工程規格與標準流 程 (Conformance)	遵循 Clean Code 規範、零規格偏 離、通過 Quality Gate	規格本身有漏洞 時，做出一套完美 的垃圾
產品觀點 (Product View)	產品本身的內在技術特 性與架構材質	高內聚低耦合、強 固的型態系統、低 圈複雜度	架構腐化，改一個 小功能引發全面崩 潰
價值觀點 (Value-based View)	顧客願意支付的成本與 性價比 (ROI)	軟體帶來的商業價 值大於開發與維運 成本	開發成本失控超 支，商業上不可行

👍 程式必須是為了給人看而寫，命令機器執行只是附帶任務。—— Abelson & Sussman
 👍 品質不是動作，是一種習慣。—— Aristotle

概念核對問答 (CCQ 2)

問題

某專案團隊開發的電商 App 完全符合合約規格書上的每一條需求（製造觀點合格），但因為底層架構高度耦合且完全沒有寫單元測試，半年後客戶想新增一個促銷功能時，工程團隊發現必須重寫整個系統。這代表該軟體在 Garvin 的哪一個品質觀點上嚴重不及格？

- A) 產品觀點 (Product View)
- B) 製造觀點 (Manufacturing View)
- C) 法律合約觀點
- D) 超自然觀點 (Transcendental View)

► 點擊查看【概念核對問答】答案與解析

1.4 軟體品質工程核心概念：V&V、品質成本 (CoQ) 與測試左移

1.4.1 驗證與確認 (Verification vs. Validation, V&V)

軟體測試與品質保證的靈魂大問：

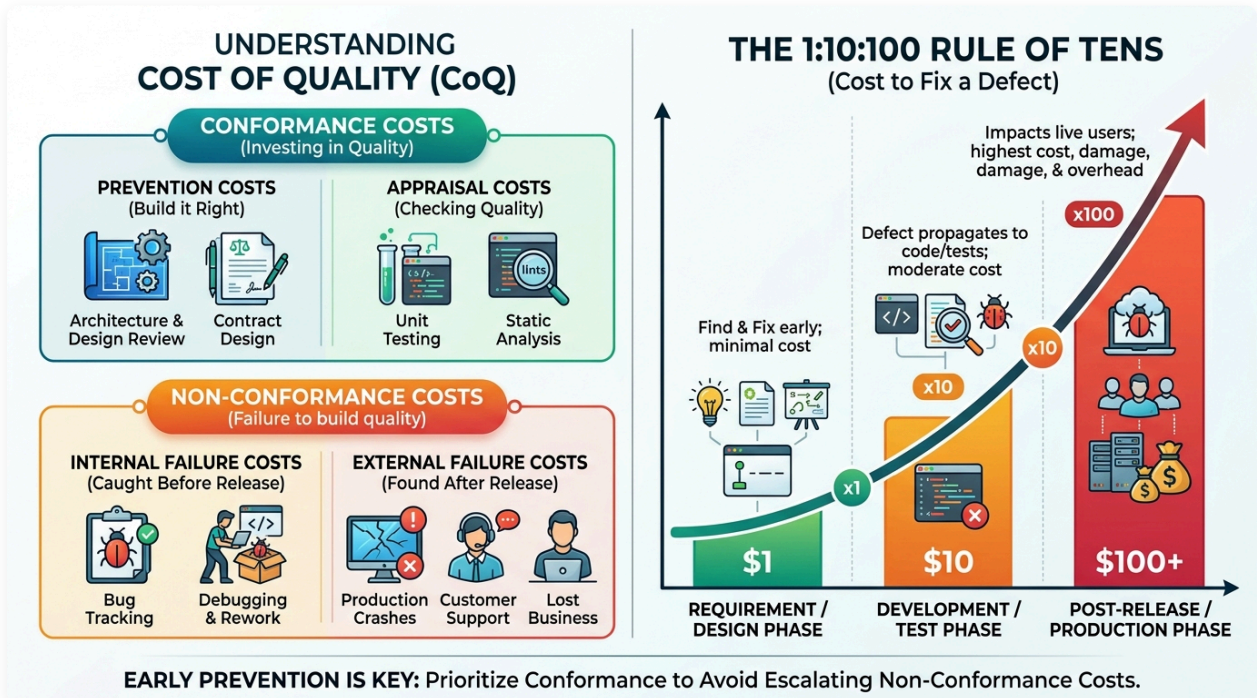
🔍 **Verification (驗證)** : *Are we building the product **right**?* (我們是否有正確地建造軟體?)

🎯 **Validation (確認)** : *Are we building the **right** product?* (我們建造的是否是正確的軟體?)

- **Verification (驗證)**：確保軟體產出物符合上個階段設定的規格（檢視程式碼是否符合設計圖、單元測試是否符合規格）。
- **Validation (確認)**：確保軟體真正滿足使用者的真實業務需求（驗收測試、易用性測試、現場 Beta 測試）。

1.4.2 軟體品質成本 (Cost of Quality, CoQ) 與 1:10:100 定律

軟體品質並不是「越完美越好」，而是在成本與效益之間取得最佳平衡。在軟體品質管理中，品質成本 (Cost of Quality, CoQ) 分為一致性成本與非一致性成本：



圖形解說：品質成本架構 (CoQ) 與 1:10:100 缺陷倍增定律

- 一致性成本 (Conformance Costs - 主動投資品質)：
 - 預防成本 (Prevention)：架構審查、合約設計 (Design by Contract)、工程培訓與靜態代碼規範。

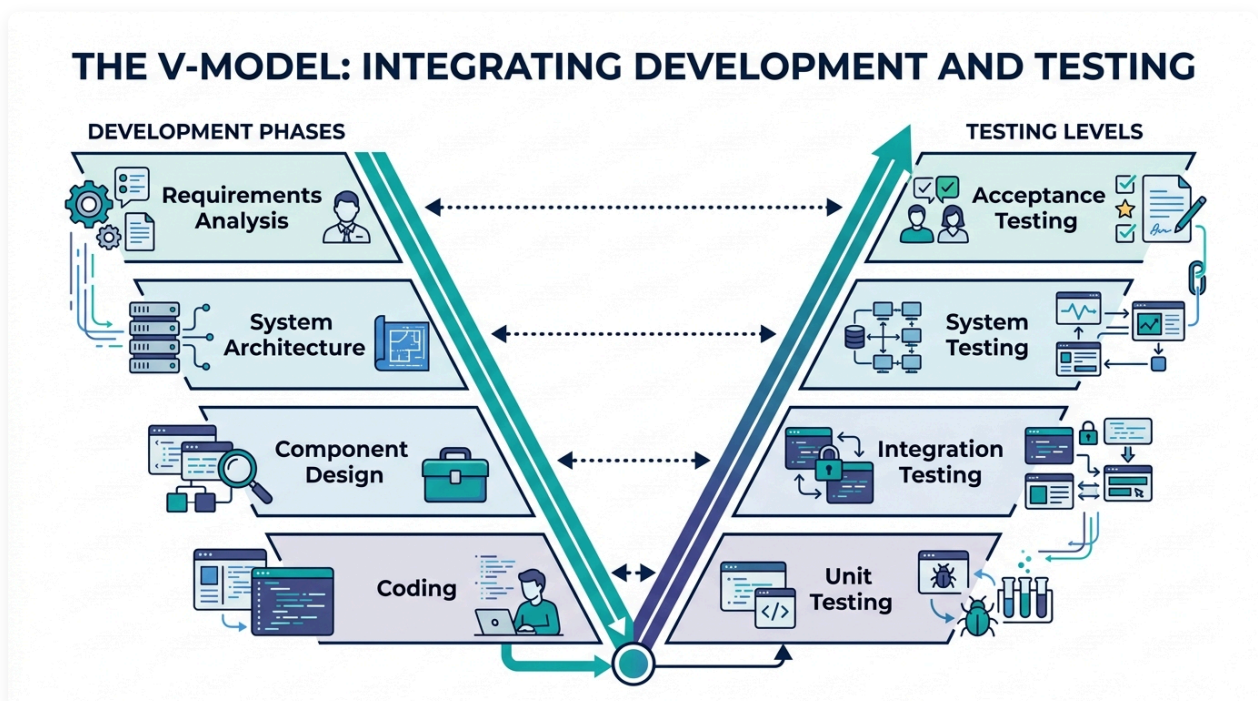
- 評估成本 (Appraisal)：單元測試 (Unit Tests)、靜態程式碼分析 (SonarQube) 與同行代碼審查 (Code Review)。
- 非一致性成本 (Non-Conformance Costs - 忽視品質的慘痛代價)：
 - 內部失敗成本 (Internal Failure)：上線前發現 Bug 導致的除錯 (Debugging)、重構與重測返工成本。
 - 外部失敗成本 (External Failure)：生產環境崩潰 (Outage)、客戶求償、緊急熱修復 (Hotfix) 與商譽破產。
- 1:10:100 定律 (The Rule of Tens)：
 - 在需求/設計階段 發現並修復缺陷的代價為 \$1。
 - 若拖延至開發/測試階段 修復代價暴增至 \$10。
 - 若洩漏至產品發布後 (Production Phase)，修復代價與災難損失將高達 100 ~ 1000+！
- 測試左移 (Shift-Left Testing)：將品質活動儘早移至生命週期前端，是降低軟體總擁有成本的最有效手段。

1.5 軟體工程流程與生命週期中的品質把關 (SDLC & CI/CD Quality Governance)

軟體工程的核心哲學在於：「品質不是最後靠測試敲打出來的，而是在整個生命週期中逐步建造並防護出來的 (Quality is built-in, not tested-in)。」

1.5.1 傳統模型與 V 模型：對稱性與早期測試規劃

在傳統線性模型（瀑布模型）中，測試常被延後至編程結束後才進行，落入 1:10:100 的高昂修復陷阱。為解決此問題，V 模型 (V-Model) 建立了開發階段與測試層級的嚴密對稱與平行規劃：

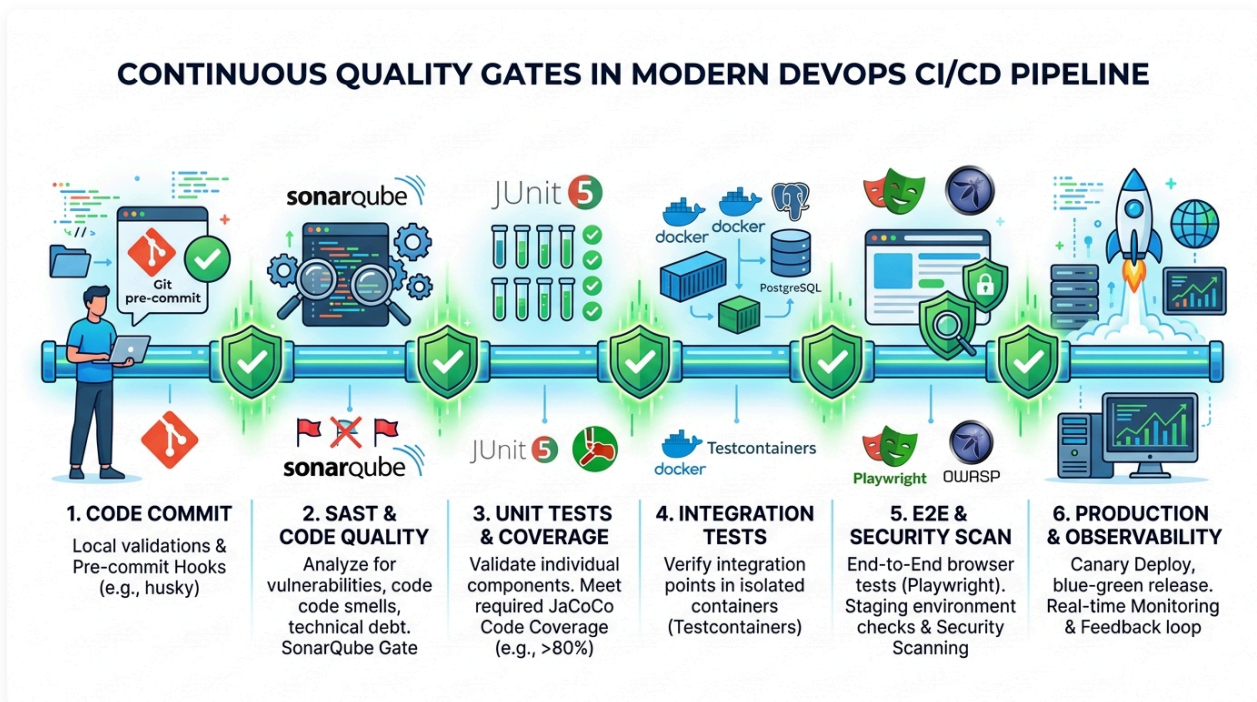


圖形解說：V 模型 (V-Model) 開發與測試層級對稱圖

- 左側：開發階段 (Verification) → 右側：測試層級 (Validation) 平行對稱：
 - i. 需求分析 (Requirements Analysis) → 同步規劃並設計 驗收測試 (Acceptance Testing)。
 - ii. 系統架構 (System Architecture) → 同步規劃並設計 系統測試 (System Testing)。
 - iii. 元件設計 (Component Design) → 同步規劃並設計 整合測試 (Integration Testing)。
 - iv. 編寫程式碼 (Coding) → 實作並執行 單元測試 (Unit Testing)。
- 核心價值：在寫下第一行業務程式碼之前，驗收與整合測試的規格與邊界就已經隨同架構圖確立完成。

1.5.2 現代敏捷與 DevOps CI/CD 連續品質門檻 (Continuous Quality Gates)

在現代雲原生與微服務時代，軟體以每日甚至每小時的頻率持續交付。品質保證已全面升級為自動化流水線上的「連續品質門檻 (Continuous Quality Gates)」：



圖形解說：現代 DevOps CI/CD 流水線中的 6 大連續品質門檻

1. **Code Commit 門檻**：本地 Git Pre-commit Hook 自動執行代碼格式化與快速靜態語法檢查。
2. **SAST 靜態程式碼品質門檻**：SonarQube / SpotBugs 掃描程式碼異味 (Code Smells)、技術債與 OWASP 安全弱點。
3. **Unit Tests & 覆蓋率門檻**：JUnit 5 執行毫秒級單元測試，並由 JaCoCo 驗證行覆蓋率與分支覆蓋率門檻（如 > 80%）。
4. **Integration Tests 容器整合門檻**：Testcontainers 一鍵拉起真實 Docker 容器 (PostgreSQL / Redis)，驗證真實資料庫存取與 API 契約。

5. **5. E2E & Security Scan 驗收門檻**：Playwright 自動化模擬真實使用者操作流程，搭配 OWASP ZAP 進行動態滲透掃描。
6. **6. Production & Observability 部署自癒門檻**：透過金絲雀 (Canary) 或藍綠部署平滑發布，並由可觀測性 (Observability) 系統即時監控 P99 延遲與異常告警。

1.6 現代軟體品質模型 (ISO 9126 → ISO 25010)

每一個產業都有其獨特的品質模型。例如製造簡易塑膠椅的廠商不會將「可維修性」列為核心品質指標，椅子壞了直接丟棄換新即可；但汽車產業就必須將「可維護性 (Maintainability)」與「安全性 (Safety)」置於最高優先級。



圖形解說：不同產業與物品具備截然不同的品質模型

1. 汽車產業 (Automobile)：優先著重於 安全性 (Safety)、可維護性 (Maintainability) 與 抗損耐用度 (Repairability)。
2. 精品機械錶 (Luxury Mechanical Watch)：優先著重於 走時精確度 (Precision)、工藝品質 (Craftsmanship) 與 超自然美感 (Transcendental Elegance)。
3. 速食餐廳 (Fast Food Restaurant)：優先著重於 出餐速度 (Speed)、口味一致性 (Consistency) 與 性價比 (Value)。
4. 軟體系統 (Software Systems)：優先著重於 高併發擴充性 (Scalability)、資訊安全 (Security)、跨平台移植性 (Portability) 與 容錯自癒力 (Fault Tolerance)。

1.6.1 ISO 25010 八大產品品質特性 (Product Quality Characteristics)

國際標準組織早期制定了 ISO 9126 (定義 6 大特性)；現代 ISO 25010 (SQaRE, 軟體產品品質要求與評估標準) 進一步擴展為 8 大產品品質特性 (Product Quality)：



圖形解說：ISO 25010 八大產品品質特性（第一層核心維度）

1. 功能適合性 (Functional Suitability)

系統所提供的功能是否滿足明訂與隱含的業務需求：

- **功能完備性 (Completeness)**：功能涵蓋了所有指定的任務與使用者目標。
- **功能正確性 (Correctness / Accurateness)**：系統產出精確無誤的結果（例如：ATM 提款功能不僅能吐鈔，且扣款金額與吐鈔張數必須分毫不差）。
- **功能適切性 (Appropriateness / Suitability)**：功能是否符合軟體本質定位（例如：一款純文字 Markdown 筆記軟體若强行塞入線上聊天與直播功能，即違反適切性）。

2. 可靠性 (Reliability)

系統在特定條件與時限內維持指定效能水準的能力：

- **成熟度 (Maturity)**：在正常運作下避免發生故障的能力（低故障率）。
- **容錯度 (Fault Tolerance)**：當面對非法輸入、硬體異常或網路抖動時，系統依然能正常運作而不崩潰（例如：接收到畸形 JSON 時拋出友好提示而非直接當機）。
- **可回復性 (Recoverability)**：發生故障中斷後，重新建立服務並復原受影響資料的能力（例如：資料庫當機重啟後透過 WAL 日誌在秒級內回復資料一致性）。

3. 效能效率 (Performance Efficiency)

系統在特定條件下所展現的效能與資源消耗比例：

- **時間行為 (Time Behavior)**：系統處理請求的反應時間、延遲與吞吐量（例如：API P99 響應時間 < 200ms）。
- **資源利用率 (Resource Utilization)**：執行時所消耗的 CPU、記憶體、硬碟 I/O 與網路頻寬數量。
- **容量 (Capacity)**：系統能支援的最大並發連線數或資料儲存上限。

4. 易用性 (Usability)

使用者學習、操作與喜愛該系統容易程度：

- **易識別性 (Appropriateness Recognizability)**：使用者能否一眼看出該軟體是否符合其需求。
- **易學習性 (Learnability)**：新手使用者需要多少時間才能熟練掌握基本操作。
- **易操作性 (Operability)**：系統控制與操作是否直覺、流暢。
- **使用者錯誤防護 (User Error Protection)**：在使用者進行高危險操作前給予警告或確認（例如：格式化磁碟前跳出二次確認視窗）。

5. 安全性 (Security)

系統保護資訊與資料免受惡意攻擊與未授權存取的能力：

- **機密性 (Confidentiality)**：確保只有獲得授權的人員能存取敏感資料（如密碼加鹽雜湊存儲、傳輸全面 HTTPS 加密）。
- **完整性 (Integrity)**：防止未授權的修改或竄改。
- **抗抵賴性 (Non-repudiation)**：能證明特定動作或交易確實由特定人員發起（如數位簽章與不可篡改的稽核日誌）。
- **真實性 (Authenticity) 與 授權能力 (Accountability)**。

6. 可維護性 (Maintainability)

工程團隊修改、優化、修復或調適軟體的有效性與效率：

- **模組化 (Modularity)**：軟體由相對獨立的模組構成，修改單一模組不會對其他模組造成不可預期的連鎖破壞。
- **可分析性 (Analyzability)**：診斷軟體缺陷或評估修改影響的難易程度（例如：具備良好的結構化 Logging 與分散式追蹤 Tracing）。
- **可修改性 (Modifiability)**：在不降低整體品質的前提下修改程式碼的難易度。
- **可測試性 (Testability)**：為軟體建立測試並執行驗證的難易程度（高內聚低耦合的架構具備極高的可測試性）。

7. 可移植性 (Portability)

系統從一個硬體、軟體或作業環境轉移至另一環境的適應能力：

- **適應性 (Adaptability)**：在無需進行額外開發下適應不同作業系統或雲端平台的能力。
- **易安裝性 (Installability)**：在指定環境下成功部署並運行的簡易度。
- **易置換性 (Replaceability)**：在相同環境下替換同類軟體產品的能力（例如：使用 Docker 容器與 Testcontainers 實現開發機、CI 伺服器與生產環境的 100% 一致性）。

8. 相容性 (Compatibility)

系統與其他軟體產品在共享硬體或網路環境時的共處與互動能力：

- **共存性 (Co-existence)**：與其他獨立軟體共享公共資源（如記憶體、通訊埠）而互不干擾。
- **互通性 (Interoperability)**：透過標準協定或 API（如 REST / GraphQL / LDAP）與外部系統順暢交換資訊並協同作業。

1.6.2 ISO 25010 品質特性與 16 週實戰測試技術地圖

軟體測試絕非盲目敲打程式碼，本課程所教授的每一項測試與工程技術，都是在為品質模型的特定維度建立自動化守護防線：

ISO 25010 品質特性	核心子特性 (Sub-characteristics)	本課程對應之測試與工程技術
功能適合性 (Functional Suitability)	完備性、正確性、適切性	等價類分割 (EP)、邊界值分析 (BVA)、JUnit 5、BDD (Cucumber)
可靠性 (Reliability)	成熟度、容錯度 (Fault Tolerance)、可回復性	斷言 (Assertions)、屬性測試 (jqwik Property-Based Testing)、混沌工程 (Chaos)
可維護性 (Maintainability)	模組化、可分析性、可修改性、可測試性	靜態程式碼分析 (SonarQube/SpotBugs)、變異測試 (PITest)、依賴解耦
安全性 (Security)	機密性、完整性、抗抵賴性、真實性	靜態安全掃描 (AST/SAST)、模糊測試 (Fuzzing with Jazzer)
效能效率 (Performance Efficiency)	時間行為 (延遲/回應時間)、資源利用率	k6 / Apache JMeter 高併發壓測、GC 監控與記憶體洩漏分析
相容性 (Compatibility)	共存性、互通性 (Interoperability)	微服務契約測試 (Pact)、跨版本相容性測試
可移植性 (Portability)	適應性、易安裝性、易置換性	Testcontainers 容器化測試、雲原生多環境測試
易用性 (Usability)	易識別性、易學習性、易操作性、錯誤保護	Playwright E2E 驗收測試、使用者流程自動化驗證

1.7 綜合練習與思維激盪

1. AI 時代的品質反思：

- 當生成式 AI 可以在幾秒鐘內產生包含完整 Javadoc 的程式碼時，為什麼軟體測試工程師的價值反而大幅提升？請從「Test Oracle 問題」與「自我印證偏誤」兩方面進行說明。

2. ISO 25010 維度分析：

- 請分析「微服務系統在後端資料庫當機重啟後，能在 5 秒內自動重新連線並將失敗的訊息重試成功，完全不丟失任何交易」，這體現了 ISO 25010 中的哪些品質特性？

3. 愛國者飛彈與數值精度：

- 試寫一小段 Java 程式碼，連續將 0.1 累加 1,000,000 次，比較其結果與 100000.0 的差異。觀察浮點數在長時間累計下的偏差現象。

